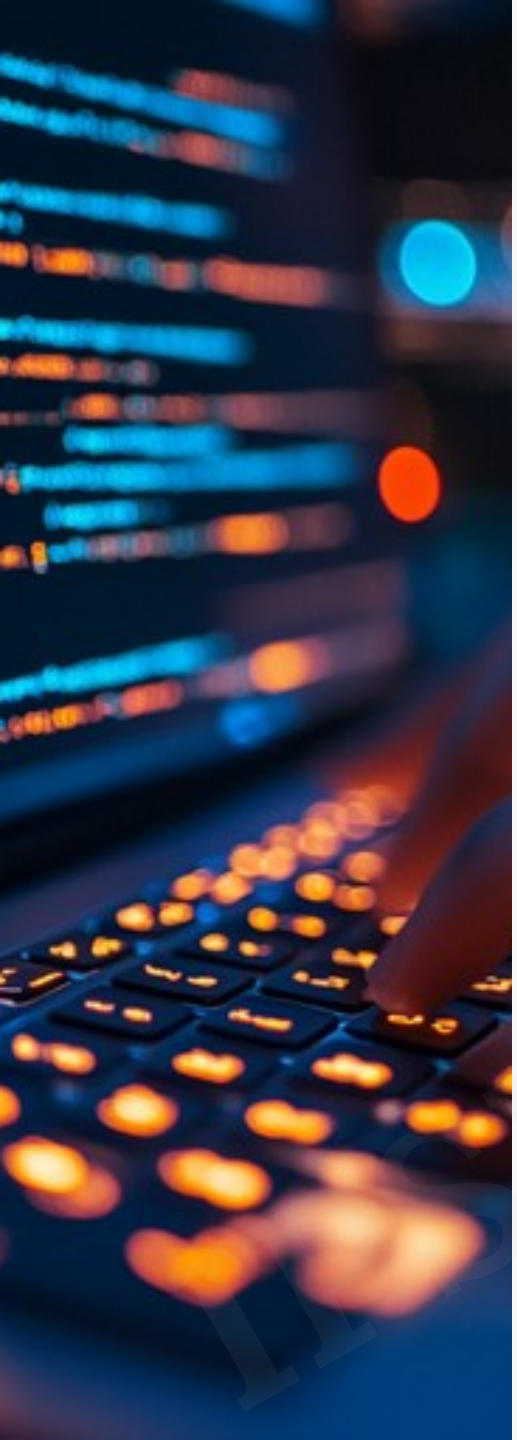


第 3 章、組合語言

CPU0 的組合語言

李建緯
雲科大資工系 智慧感知與系統整合實驗室
IPSI Lab, Yuntech-CSIE

IPSI Lab, Yuntech-CSIE

CPU0 的組合語言

- 前置表示法
  - CPU0 的組合語言一律採用目標在前的撰寫方式。

組合語言

```
ADD R1, R2, R3
```

C語言

```
R1 = R2+R3;
```

IPSI Lab, yuntech-CSIE

基本範例

資料移動

IPSI Lab, Yuntech-CSIE

C語言中的資料移動語句

```c
void main() {
    int x = 3, y;
    y = x;
}
```

- 這段程式將 `x` 的值 3 指定給 `y`，最終 `y` 的值為 3。

IPSI Lab, Yuntech-CSIE

組合語言中的資料移動指令

暫存器之間的資料移動

MOV 指令

- 在 CPU0 的組合語言中，與 C語言類似，資料移動使用 MOV 指令。
- MOV 指令通常將資料從一個暫存器移動到另一個暫存器。

範例

(a) 組合語言	(b) C 語言
`MOV R1, R2`	R1 = R2

IPSI Lab, Yuntech-CSIE

記憶體與暫存器之間的資料移動

LD 與 ST 指令

- 當需要將資料從記憶體移動到暫存器時，使用 `LD` (載入) 指令。
- 反之，將資料從暫存器移回記憶體時使用 `ST` (儲存) 指令。

範例：以組合語言模擬 C 語言當中的指定語句 A = B

(a) 組合語言	(b) C 語言（對照版）	(c) C 語言（簡化版）
`LD R1, B`	R1 = B	A = B
`ST R1, A`	A = R1	

IPSI Lab, Yuntech-CSIE

基本數學運算

IPSI Lab, Yuntech-CSIE

組合語言中的加減運算

- 除了資料移動之外，組合語言可以處理基本的數學運算，包括加法、減法、乘法、除法。
- 數學運算通常是在暫存器中進行，因此在進行運算之前，必須先將記憶體中的資料載入暫存器。

IPSI Lab, Yuntech-CSIE

C語言中的 A = B + C - D

運算步驟

- C語言的運算式 `A = B + C - D` 需要分別將 `B`, `C`, `D` 載入暫存器，然後進行加法與減法運算。

IPSI Lab, Yuntech-CSIE

範例：以組合語言模擬 C 語言當中的數學運算式 A = B + C - D

(a) 組合語言

```
LD   R2, B
LD   R3, C
LD   R4, D
ADD  R1, R2, R3
SUB  R1, R1, R4
ST   R1, A
```

(b) C 語言（對照版）

```
R2 = B;
R3 = C;
R4 = D;
R1 = R2 + R3;
R1 = R1 - R4;
A  = R1;
```

(c) C 語言（簡化版）

```
A = B + C - D;
```

IPSI Lab, Yuntech-CSIE

模擬條件判斷

IPSI Lab, Yuntech-CSIE

組合語言中的條件跳躍

C語言中的 if 判斷式

- 在 C 語言中，`if` 是一個很重要的條件判斷指令。
- 組合語言中沒有 `if` 指令，但可以使用 `CMP` 比較指令與 `JGT`、`JEQ` 等跳躍指令來模擬 `if` 的功能。

IPSI Lab, Yuntech-CSIE

模擬條件判斷 - 以 if (A > B) 為例

- 假設我們要模擬 C 語言中的 `if (A > B)`，組合語言可以透過 `CMP` 進行比較，並使用 `JGT` 來跳躍至標記。

組合語言流程

```
    LD R1, A      ; 將 A 載入 R1
    LD R2, B      ; 將 B 載入 R2
    CMP R1, R2    ; 比較 R1 與 R2 (A 與 B)
    JGT IF        ; 若 R1 > R2，跳到標記 IF
    ST R2, C      ; 若 R1 <= R2，將 B (R2) 存入 C
    JMP EXIT      ; 跳到結束

IF:               ; 標記 IF
    ST R1, C      ; 將 A (R1) 存入 C
EXIT:             ; 結束點
    RET           ; 返回
```

IPSI Lab, Yuntech-CSIE

指令說明

1. `LD R1, A`：將變數 A 載入暫存器 R1。
2. `CMP R1, R2`：比較 R1 與 R2 的值。
3. `JGT IF`：如果 R1 大於 R2，跳躍到標記 IF。
4. `ST R1, C`：根據判斷結果，將 A 或 B 存入變數 C。

IPSI Lab, Yuntech-CSIE

範例 3.5 以組合語言模擬 C 語言當中的 if 判斷式

(a) 組合語言

```
    LD   R1, A
    LD   R2, B
    CMP  R1, R2
    JGT  IF
    ST   R2, C
    JMP  EXIT
IF:
    ST   R1, C
EXIT:
    RET
```

(b) C 語言（對照版）

```
      R1 = A;
      R2 = B;
      if (R1 > R2)
        goto IF;
      C = R2;
      goto EXIT;
IF:
      C = R1;
EXIT:
      RET;
```

(c) C 語言（簡化版）

```
if (A > B)
  C = A;
else
  C = B;

return;
```

IPSI Lab, Yuntech-CSIE

模擬迴圈

IPSI Lab, Yuntech-CSIE

組合語言中的迴圈

- 在 C 語言中，迴圈是語法上較為複雜的指令。
- 組合語言中沒有原生的迴圈指令，需要透過**跳躍指令來模擬迴圈**的功能。

組合語言模擬無窮迴圈

```
LOOP:
    ADD R1, R1, R2   ; R1 = R1 + R2
JMP LOOP             ; 無窮迴圈，跳回 LOOP
```

C語言（對照版）

```
while (1) {
  R1 = R1 + R2;
}
```

IPSI Lab, Yuntech-CSIE

宣告變數與初始值

IPSI Lab, Yuntech-CSIE

範例 3.7：組合語言的完整程式範例，加總迴圈，從 1 加到 10

(a) 組合語言

```
    LD   R1, sum
    LD   R2, i
    LDI  R3, 10
    LDI  R4, 1
FOR:
    CMP  R2, R3
    JGT  EXIT
    ADD  R1, R2, R1
    ADD  R2, R4, R2
    JMP  FOR
EXIT: RET

i:    RESW 1
sum:  WORD 0
```

(b) C 語言（對照版）

```
R1 = sum;
R2 = i;
R3 = 10;
R4 = 1;

if (R2 > R3)   // (i > 10)
    goto EXIT;

R1 = R1 + R2;
R2 = R2 + R4;
goto FOR;

EXIT: return;

int i;
int sum = 0;
```

(c) C 語言（真實版）

```
int sum = 0;
int i;

for (i = 1; i <= 10; i++)
    sum += i;
```

組合語言中的宣告變數與初始值

- **在 CPU0 的組合語言中，若要宣告變數，可使用 `RESW`、`RESB`、`WORD`、`BYTE` 等指令來保留或初始化存儲空間。**
- **範例 3.7 中：**
  - `RESW 1` 為變數 `i` 保留 1 個字組 (Word) 空間。
  - `sum: WORD 0` 則為變數 `sum` 設定初始值 0。
- **`RESW`（Reserve Word）：保留一個或多個字（word）的空間，無提供初始化。每個字通常是 4 個位元組（32 位元）。**
  - 範例：`i: RESW 1`，保留一個字的空間給變數 `i`。

- **`RESB`（Reserve Byte）：保留一個或多個位元組的空間，無提供初始化。**
  - 範例：`buffer: RESB 10`，保留 10 個位元組的空間給變數 `buffer`。
- **`WORD`：用來初始化一個字的空間並賦初始值。**
  - 範例：`sum: WORD 0`，宣告變數 `sum` 並初始化為 0。
- **`BYTE`：用來初始化一個位元組的空間並賦初始值。**
  - 範例：`charVar: BYTE 65`，宣告一個位元組大小的變數 `charVar` 並初始化為 65（即 ASCII 的 'A'）。

模擬有條件的迴圈

- 若要模擬一個有條件的迴圈，組合語言需要配合比較指令與條件跳躍指令來實現。

加總 1 到 10

```
int sum = 0;
int i;
for (i = 1; i <= 10; i++) {
    sum += i;
}
return;
```

IPSI Lab, yuntech-CSIE

組合語言模擬 for 迴圈

```
    LD R1, sum      ; 將 sum 載入 R1
    LD R2, i        ; 將 i 載入 R2
    LDI R3, 10      ; 將常數 10 載入 R3
    LDI R4, 1       ; 將常數 1 載入 R4

FOR:
    CMP R2, R3      ; 比較 R2 和 R3
    JGT EXIT        ; 若 i > 10，跳至 EXIT
    ADD R1, R2, R1  ; sum += i
    ADD R2, R4, R2  ; i++
    JMP FOR         ; 跳回 FOR 繼續迴圈

EXIT: RET           ; 結束程序
```

C 語言對應程式碼

```
R1 = sum;
R2 = i;
R3 = 10;
R4 = 1;

FOR:
    if (R2 > R3) goto EXIT; //判斷是否跳至 EXIT
    R1 = R1 + R2;
    R2 = R2 + R4;
    goto FOR;
EXIT: return;
```

IPSI Lab Yuntech-CSIE

陣列存取

陣列存取

- 如何使用 CPU0 中的相對定址方式來進行字串複製。
- 模擬兩種 C 語言的實作方式：

1. **指標操作**：將字串透過指標進行複製。
2. **陣列操作**：透過陣列索引來複製字串。

IPSI Lab, Yuntech-CSIE

字串（字元陣列）

IPSI Lab, Yuntech-CSIE

C 語言中字串的存放方式

```c
char b[] = "Hello!";
```

- 字串在記憶體中的結構：

Index	0	1	2	3	4	5	6
Value	'H'	'e'	'l'	'l'	'o'	'!'	'\0'

- 字串的每個字元依次存放在記憶體中，每個字元佔用 **1 byte**。
- 字串的最後會自動加上一個 `'\0'`（空字元），用來標示**字串的結束**。

IPSI Lab, Yuntech-CSIE

記憶體存放的樣貌

- 假設記憶體起始地址是 `0x1000`，在記憶體中可能排列方式。
- 每個字元佔用 1 byte，並且是連續存放的。

Address	Value
0x1000	'H'
0x1001	'e'
0x1002	'l'
0x1003	'l'
0x1004	'o'
0x1005	'!'
0x1006	'\0'

IPSI Lab, Yuntech-CSIE

C 語言字串的讀取方式

透過指標

```
char *ptr = b;
while (*ptr != '\0') {
    putchar(*ptr);
    ptr++;
}
```

- 使用指標 `ptr` 指向字串 `b` 的起始位置。
- 然後透過 `*ptr` 讀取當前指向的字元，每次將指標增加 1，直到遇到結束符號 `'\0'`。

IPSI Lab, Yuntech-CSIE

透過索引

```
for (int i = 0; b[i] != '\0'; i++) {
    putchar(b[i]);
}
```

- 使用索引 `i` 來讀取字串 `b` 中的每個字元。
- `b[i]` 會從 `b` 的起始地址開始，每次讀取 1 byte，直到遇到 `'\0'`。

IPSI Lab, Yuntech-CSIE

字串複製- 指標版 (範例3.8)

C 語言範例

```c
// 將字串 'b' 中的內容複製到字串 'a' 中，
// 直到遇到字串的結束符號 '\0' 為止。
// '\0' 是 C 語言中用來標示字串結束的特殊字元。
char a[10];
char b[] = "Hello !";
char *aptr = a, *bptr = b;

while (1) {
    *aptr = *bptr;

    aptr++;
    bptr++;
    if (*bptr == '\0')
        break;
}
return;
```

IPSI Lab, Yuntech-CSIE

組合語言範例

1. **載入變數**：
   - `LD R1, i`：將索引變數 `i` 載入暫存器。
   - `LD R2, aptr` 和 `LD R3, bptr`：分別載入來源字串與目標字串的**地址**。
2. **字串複製迴圈**：
   - `LDB R4, [R3]`：從來源字串讀取當前字元。
   - `STB R4, [R2]`：將讀取的字元寫入目標字串。
   - 每次迴圈後，**遞增**指標並檢查是否遇到字串結束符號 `\0`。

IPSI Lab, Yuntech-CSIE

組合語言

```
LD R1, i          ; 載入索引變數 i
LD R2, aptr       ; 載入來源字串地址
LD R3, bptr       ; 載入目標字串地址
LDI R7, 1         ; 設定步長
while:
    LDB R4, [R3]     ; 讀取來源字串字元
    STB R4, [R2]     ; 將字元寫入目標字串
    ADD R2, R2, R7   ; 移動指標
    ADD R3, R3, R7
    CMP R4, R0       ; 判斷是否為 '\0'
    JEQ endw         ; 如果是，結束
    JMP while        ; 否則繼續迴圈
endw:
    RET              ; 返回

a:    RESB 10
b:    BYTE "Hello !", 0
aptr: WORD a
bptr: WORD b
i:    WORD 0
```

C語言對應程式碼

```
R1 = i;                 // 初始化索引變數
R2 = *aptr;             // 來源字串指標
R3 = *bptr;             // 目標字串指標
R7 = 1;                 // 步長

while {
    R4 = [R3];          // 從 bptr 讀取字元
    [R2] = R4;          // 將字元寫入 aptr
    R2 = R2 + R7;       // 更新目標指標
    R3 = R3 + R7;       // 更新來源指標
    if (R4 == '\0') // 如果遇到 '\0'，則跳出
      goto endw;
    goto while;
}
endw:
return;

char a[10];
char b[] = "Hello !"
int  *aptr=a;
int  *bptr=b;
int  i=10;
```

▶範例 3.8 字串複製 (指標版)

(a) 組合語言

```
    LD   R1, i
    LD   R2, aptr
    LD   R3, bptr
    LDI  R7, 1
while:
    LDB  R4, [R3]
    STB  R4, [R2]
    ADD  R2, R2, R7;
    ADD  R3, R3, R7;
    CMP  R4, R0
    JEQ  endw
    JMP  while
endw:
    RET
a:      RESB 10
b:      BYTE "Hello !", 0
i:      WORD 0
aptr:   WORD a
bptr:   WORD b
```

(b) C 語言 (對照版)

```
R1=i;
R2=*aptr;
R3=*bptr;
R7=1;
while:
   R4 = [R3]; // R4 = *bptr
   [R2] = R4; //*aptr = R4
   R2= R2+1;
   R3=R3+1;
   if (R4!= R0) // b[i] != '\0'
   goto endw;
   goto while;
endw:
      return;
char a[10];
char b[] = "Hello !"
int i=0;
int *aptr = a;
int *bptr = b;
```

(c) C 語言 (真實版)

```
char a[10];
char b[] = "Hello !";
char *aptr=a, *bptr=b;

while (1) {
   *aptr = *bptr;

   aptr++;
   bptr++;
   if (*bptr == '\0')
      break;
}
return;
```

字串複製- 索引版（範例3.9）

C 語言範例

```c
// 使用陣列索引進行字串的逐字元複製

char a[10];
char b[] = "Hello!";
int i = 0;

while (1) {
    a[i] = b[i];
    if (b[i] == '\0')
      break;
    i++;
}
return
```

IPSI Lab, Yuntech-CSIE

組合語言範例

1. **載入變數**：
   - `LD R1, i`：將索引變數 `i` 載入暫存器。
   - `LD R2, aptr`：載入來源字串地址。
   - `LD R3, bptr`：載入目標字串地址。
2. **字串複製迴圈**：
   - `LBR R4, [R3+R1]`：讀取來源字串的字元。
   - `SBR R4, [R2+R1]`：將字元寫入目標字串。
   - 每次迴圈後遞增索引 `R1`，並通過 `CMP` 指令判斷是否到達 `\0` 字元，若是則退出迴圈。

IPSI Lab, Yuntech-CSIE

組合語言

```
    LD R1, i          ; 載入索引變數 i
    LD R2, aptr       ; 載入來源字串地址
    LD R3, bptr       ; 載入目標字串地址
    LDI R7, 1         ; 設定遞增值

while:
    LBR R4, [R3+R1]     ; 讀取來源字串 b[i]
    SBR R4, [R2+R1]     ; 將字元寫入目標字串 a[i]
    CMP R4, R0          ; 判斷是否遇到 '\0'
    JEQ endw            ; 若遇到 '\0'，跳轉至 endw
    ADD R1, R1, R7      ; 索引增加
    JMP while           ; 繼續迴圈
endw:
    RET                 ; 結束返回

a:    RESB 10
b:    BYTE "Hello !", 0
aptr: WORD a
bptr: WORD b
i:    WORD 0
```

C 語言對應程式碼

```
R1 = i;                 // 索引變數
R2 = *aptr;             // 來源字串地址
R3 = *bptr;             // 目標字串地址
R7 = 1;                 // 步長

while {
    R4 = [R3+R1];       // 讀取來源字串 b[i]
    [R2+R1] = R4;       // 將字元寫入目標字串 a[i]
    if (R4 != R0)       // 判斷是否遇到 '\0'
        goto endw;
    R1 = R1 + R7;       // 索引加 1
        goto while
}
endw:
return;

char a[10];
char b[] = "Hello !"
int  *aptr=a;
int  *bptr=b;
int  i=10;
```

IPSI Lab, Yuntech-CSIE

▶範例 3.9 字串複製 (索引版)

(a) 組合語言

```
        LD   R1, i
        LD   R2, aptr
        LD   R3, bptr
        LDI  R7, 1
while:
        LBR  R4, [R3+R1]
        SBR  R4, [R2+R1]
        CMP  R4, R0
        JEQ  endw
        ADD  R1, R1, R7
        JMP  while
endw:   RET
a:      RESB 10
b:      BYTE "Hello !", 0
aptr:   WORD a
bptr:   WORD b
i:      WORD 0
```

(b) C 語言 (對照版)

```
R1=i;
R2=*aptr;
R3=*bptr;
R7=1;
while:
  R4 = [R3+R1];    // R4 = b[i]
  [R2+R1] = R4;    // a[i] = R4
  if (R4!= R0)     // b[i] != '\0'
    goto endw;
  R1 = R1 + R7; // i++;
    goto while;
endw: return;
char a[10];
char b[] = "Hello !"
int *aptr=a;
int *bptr=b;
int i=0;
```

(c) C 語言 (真實版)

```
char a[10];
char b[] = "Hello !";
int i = 0;

while (1) {
  a[i] = b[i];

  if (b[i] == '\0')
    break;
  i++;
}
return;
```

整數陣列

IPSI Lab, Yuntech-CSIE

整數陣列的存放方式

```
int arr[] = {1, 2, 3, 4, 5};
```

- 假設 `int` 在這個系統中是 **4 bytes**，整數陣列 `arr` 在記憶體中的結構：

Index	0	1	2	3	4
Value	1	2	3	4	5

- 每個整數佔用 **4 bytes**，所以在記憶體中這些整數是連續存放的。

IPSI Lab, Yuntech-CSIE

記憶體存放的樣貌

- 假設記憶體起始地址是 `0x2000`，那麼整數陣列 `arr` 在記憶體中可能的排列方式。
- 每個整數佔用 **4 bytes**，因此每個整數之間的地址相隔 4 bytes。

Address	Value
0x2000	1
0x2004	2
0x2008	3
0x200C	4
0x2010	5

IPSI Lab, Yuntech-CSIE

整數陣列的讀取方式

透過索引讀取

```
for (int i = 0; i < 5; i++) {
    printf("%d ", arr[i]);
}
```

- `arr[i]` 用來讀取陣列中的第 i 個整數。
- 編譯器會自動將 `arr[i]` 轉換為：

  ```
  *(arr + i)
  ```

  也就是：
  從陣列起始位址偏移 `i * sizeof(int)` 個位元組（bytes），再讀取該位置的整數。

透過指標讀取

```
int *ptr = arr;
for (int i = 0; i < 5; i++) {
    printf("%d ", *(ptr + i));
}
```

- `ptr` 是一個指向整數陣列 `arr` 起始位址的指標。
- `*(ptr + i)` 代表：取出從起始位址向後偏移 i 個整數位置的值。
- 因為 `ptr` 是 `int*` 型別，指標運算會自動以 `sizeof(int)` 為單位進行位址遞增。(Refer to S03_03-CPU0_Assembly_Lang-補充C指標)

IPSI Lab, Yuntech-CSIE

整數陣列的存放與字元陣列的區別

- **單位大小**：字元陣列中的每個元素（字元）佔用 1 byte，而整數陣列中的每個元素（整數）佔用多個 bytes（通常是 4 bytes 或 2 bytes，取決於系統架構）。
- **記憶體地址的間隔**：在字元陣列中，每個字元之間的地址相差 1 byte；而在整數陣列中，每個整數之間的地址相差 `sizeof(int)`（通常是 4 bytes）。

IPSI Lab, Yuntech-CSIE

整數陣列與字元陣列的記憶體差異

比較項目	整數陣列（`int arr[]`）	字元陣列（`char arr[]`）
元素大小	通常為 4 bytes（依平台而異）	每個字元為 1 byte
位址間隔	每兩個元素間隔 `sizeof(int)`	每兩個字元間隔 1 byte
指標遞增	`ptr + 1` → 下一個整數位置	`ptr + 1` → 下一個字元位置
主要用途	儲存整數資料	儲存文字或字串

IPSI Lab, Yuntech-CSIE

例：位址變化對照

假設 `arr` 的起始位址為 `0x1000`：

索引	`arr[i]`	位址	若為 `char arr` 時的位址
0	arr[0]	0x1000	0x1000
1	arr[1]	0x1004	0x1001
2	arr[2]	0x1008	0x1002

可以看出整數陣列每個元素之間的間距是 4 bytes，而字元陣列僅有 1 byte。

IPSI Lab, Yuntech-CSIE

整數陣列的複製（範例3.10）

```
// 將 a[i] 複製到 b[i]

int a[100], b[100];
int i;

for (i = 0; i < 100; i++) {
    b[i] = a[i];
}
```

IPSI Lab, Yuntech-CSIE

組合語言

1. **載入變數**：
   - `LD R2, aptr` 和 `LD R3, bptr`：分別載入來源和目標陣列的起始地址。
   - `LDI R8, 1` 設定步長(索引的增量)，`LDI R9, 100` 設定陣列長度。
2. **迴圈處理**：
   - 透過 `SHL R5, R1, 2`，將索引值乘以 4 來計算每個整數的位置，然後透過 `LDR` 和 `STR` 指令複製整數值。

     > **為什麼是乘以 4？原因在於上述的 `sizeof(int)`。**

   - 每次迴圈後，增加索引，並比較索引與陣列長度，若未達到結尾則繼續迴圈。

- 註:
  - 每個整數在記憶體中佔用 4 個位元組（bytes）
  - `SHL R5, R1, 2`（即將索引值 R1 左移兩位）來實現索引值乘以 4 的操作

組合語言

```
LD R2, aptr     ; 載入來源陣列地址
LD R3, bptr     ; 載入目標陣列地址
LDI R8, 1       ; 設定步長(索引的增量)
LDI R9, 100     ; 設定陣列長度

FOR:
SHL R5, R1, 2   ; 將索引乘以 4 計算位址
LDR R6, [R5+R3]; 讀取來源陣列的整數值
STR R6, [R5+R2]; 將整數值寫入目標陣列
ADD R1, R1, R8 ; 索引加 1
CMP R1, R9      ; 比較索引與陣列長度
JLE FOR         ; 若未達到結尾則繼續
RET             ; 結束返回

a:    RESW 100
b:    RESW 100
aptr: WORD a
bptr: WORD b
i:    RESW 1
```

C 語言對應程式碼

```
R2 = *aptr;           // 來源陣列地址
R3 = *bptr;           // 目標陣列地址
R8 = 1;               // 步長
R9 = 100;             // 陣列長度
i = 0;                // 初始化索引
R1 = i;

FOR:
    R5 = R1 << 2;     // 索引乘 4 計算位址
    R6 = [R5+R3];     // 讀取來源陣列 a[i]
    [R5+R3] = R6;     // 將數值寫入目標陣列 b[i]
    R1 = R1 + 1;      // 索引加 1
    if (R1 <= R9)     // 判斷是否達到陣列末尾
        goto FOR;
return;

char a[100];
char b[100];
int  *aptr=a;
int  *bptr=b;
int  i;
```

▶範例 3.10 陣列複製 (索引版)

(a) 組合語言

```
        LD   R2, aptr
        LD   R3, bptr
        LDI  R8, 1
        LDI  R9, 100
        ST   R0, i
        LD   R1, i
FOR:
        SHL  R5, R1, 2
        LDR  R6, [R5+R2]
        STR  R6, [R5+R3]
        ADD  R1, R1, R8
        CMP  R1, R9
        JLE  FOR
        RET
a:      RESW 100
b:      RESW 100
aptr:   WORD a
bptr:   WORD b
i:      RESW 1
```

(b) C 語言 (對照版)

```
  R2 = *aptr;
  R3 = *bptr;
  R8 = 1;
  R8 = 100;
  i = 0;
  R1 = i;
FOR:
  R5 = R1 << 2; // R5=i*4;
  R6 = [R5+R2];   // R6 = a[i]
  [R5+R3] = R6;   // b[i] = R6;
  R1 = R1+1;   // i++;
  If (R1 <= R9) // i<=100
     goto FOR;
  return
  int a[100];
  int b[100];
  int *aptr=a;
  int *bptr=b;
  int i;
```

(c) C 語言

```
int a[100], b[100];
int i;
for (i=0; i<=100; i++) {
 a[i] = b[i];
}
```

副程式呼叫

副程式呼叫

- **定義**: 副程式呼叫是一種讓程式暫時跳轉到另一個程式區塊（即副程式）的技術。呼叫副程式可以達到程式的重複利用、邏輯分離以及結構化設計。
- **優點**: 通過副程式呼叫，主程式可以簡化邏輯，重複使用相同的程式區塊，減少程式碼冗餘，提升維護性和可讀性。
- **應用情境**: 副程式常被應用在大型軟體系統中，如嵌入式系統、作業系統內核、驅動程式開發等。在這些系統中，對硬體操作時，經常需要通過副程式來進行特定功能呼叫，如設備初始化、資料傳輸等。

IPSI Lab, Yuntech-CSIE

副程式呼叫方法

- **單層次的副程式呼叫**
  - 參數的傳遞方法 – 使用暫存器
- **多層次的副程式呼叫**
  - 參數的傳遞方法– 使用堆疊

IPSI Lab, Yuntech-CSIE

單層次的副程式呼叫

單層次的副程式呼叫指的是只有一層的呼叫和返回關係，即在主程式中呼叫一個副程式，副程式執行完畢後返回主程式，不涉及巢狀或多層次呼叫。

- **CALL 指令的運作**: CALL 指令是處理器中的一個特殊跳躍指令，它會將當前指令的位置存入**連結暫存器**（Link Register, LR），然後跳轉到副程式的地址執行。
- **RET 指令的運作**: RET 指令是返回指令，它會將暫存器（LR）中儲存的返回點重新放回程式計數器（PC），以返回主程式。

IPSI Lab, Yuntech-CSIE

CALL 指令的執行過程

1. **PC 更新**: 當處理器擷取 CALL 指令時，首先將 PC 的值加上 4，這是因為每條指令佔用的位址是 4 個位元組。
2. **暫存器 LR 設置**: CALL 指令會將當前的 PC 值（即 CALL 指令的下一個指令地址）存入暫存器 LR，這樣在副程式執行完後，能通過 RET 返回此位址。
3. **跳轉至副程式地址**: 最後，PC 會被設置為副程式的入口地址，讓控制權轉移到副程式。

IPSI Lab, Yuntech-CSIE

(1) PC=PC + 4; 在指令擷取之後 PC 從 28 變為 32。
(2) LR = PC; 將 PC 存入到連結暫存器 LR 中。
(3) PC=PC + 30

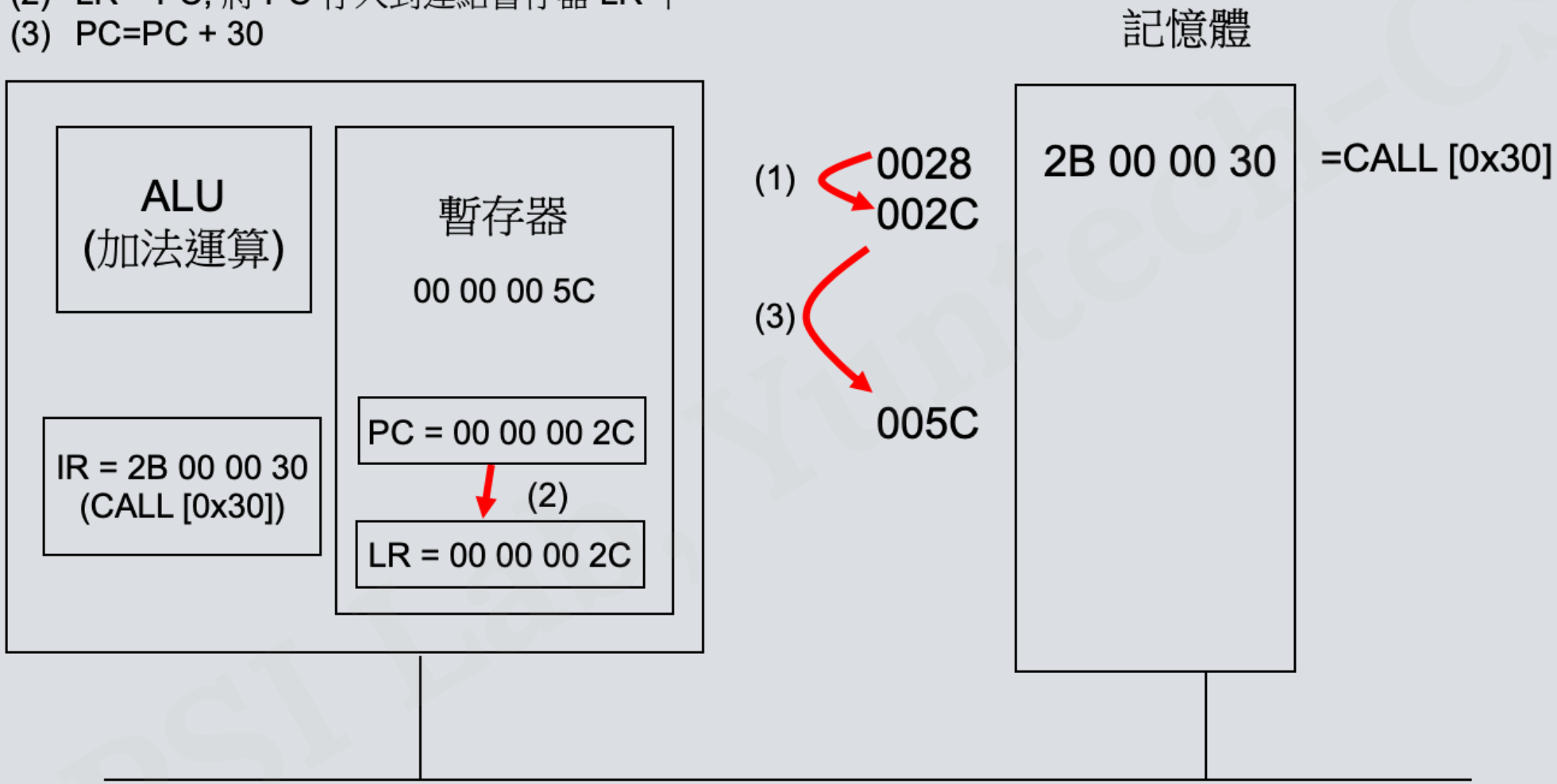


圖 3.1 指令CALL [0x30] 的執行過程

RET 指令的執行過程

1. **LR 值轉移到 PC**: 當執行 RET 指令時，處理器會將 LR 暫存器中的值複製到 PC，這樣可以讓控制權返回到主程式。
2. **繼續執行主程式**: PC 設置為之前保存的主程式的下一條指令，主程式繼續執行。

IPSI Lab, Yuntech-CSIE

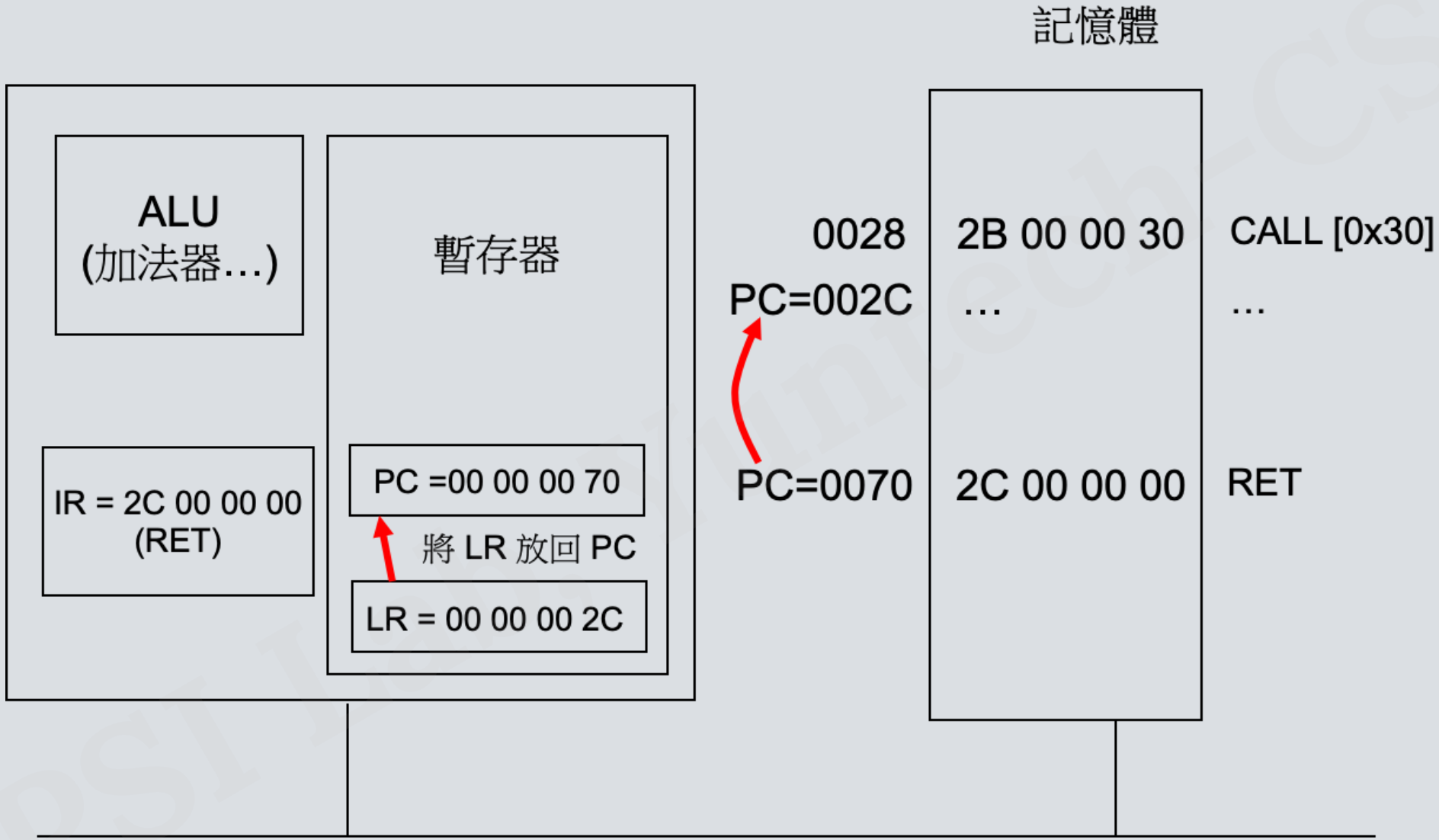


圖 3.2 指令RET 的執行過程

多層次的副程式呼叫

多層次副程式呼叫（Nested Subroutine Call）指的是副程式內部再次呼叫其他副程式，形成「巢狀」的呼叫結構。

- **呼叫**: 在多層次呼叫中，執行會從主程式跳轉到第一層副程式，然後從第一層副程式跳轉到第二層副程式，依此類推。
- **返回**: 每一層呼叫都需要正確保存返回位址，並在副程式結束時恢復返回主程式的路徑。
- **應用場景**: 多層次呼叫廣泛應用於遞迴演算法、操作系統內核設計和複雜軟體的模組化設計中。

IPSI Lab, Yuntech-CSIE

使用堆疊保存返回位址與暫存器內容

- **CALL 和 PUSH 操作**:
  - 當程式進行多層次副程式呼叫時，CALL 指令會將控制跳轉到副程式，並將返回位址（PC 值）存入暫存器 LR。
  - 若在該副程式內又進行再次呼叫另一層副程式時，LR 值會被覆蓋，導致上一層的返回位址遺失。
  - 因此在進行多層次呼叫時，**必須先用 PUSH 將 LR 壓入堆疊（Stack）**保存。
- **POP 和 RET 操作**:
  - 副程式執行完畢後，**通過 POP 指令從堆疊中彈出保存的 LR 值**，然後用 RET 指令返回到正確的上一層位置。

IPSI Lab, Yuntech-CSIE

為什麼使用堆疊傳遞參數與保存暫存器

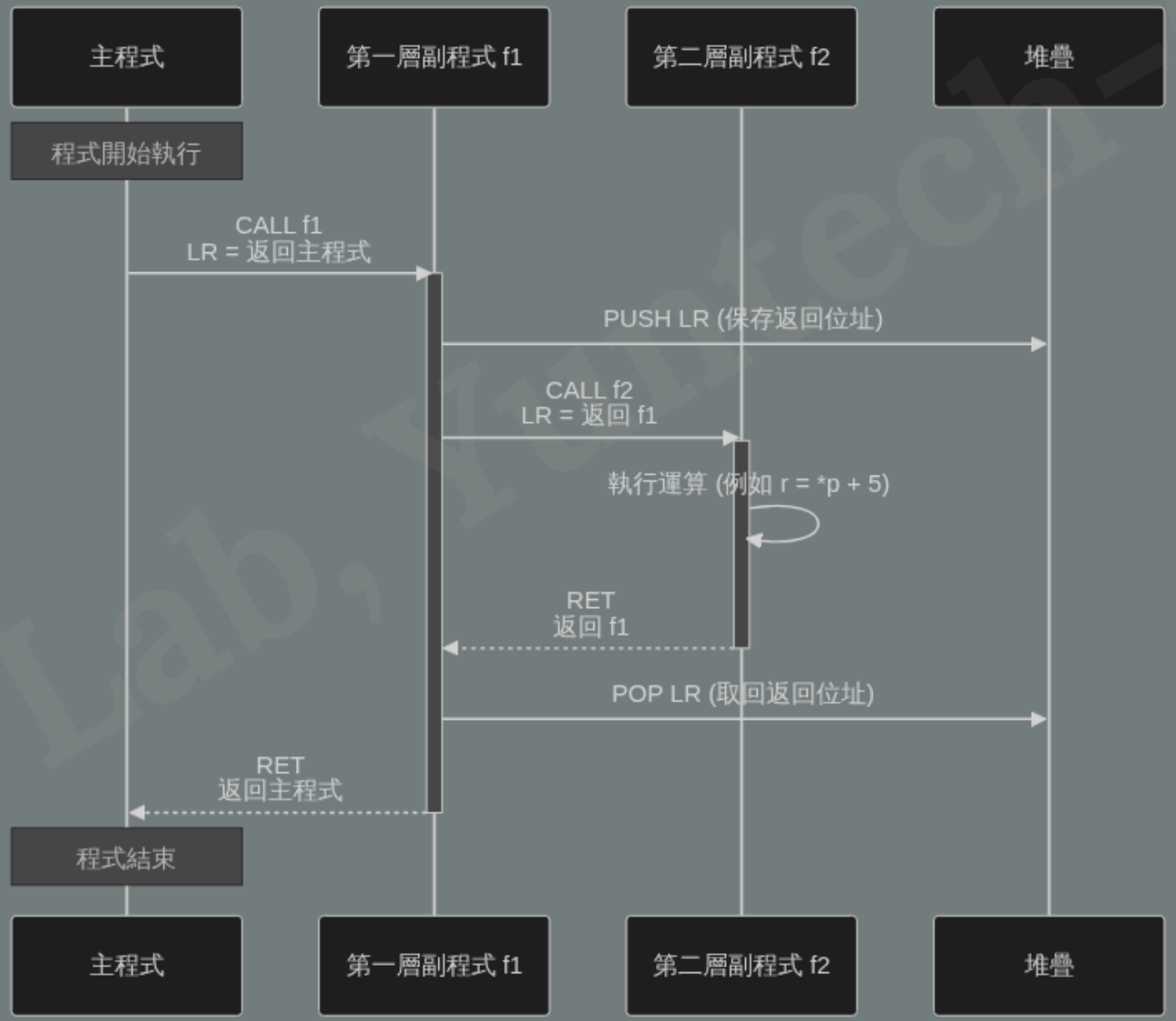
- 可避免上下層副程式使用相同暫存器而互相覆蓋。
- 將 LR 儲存在堆疊中，可防止多層 CALL 時返回位址被下一層覆寫。
- 每層呼叫皆擁有獨立的「活動紀錄（Stack Frame）」，方便遞迴與模組化管理。儲存到堆疊中，以免在下一層 CALL 返回位址被覆蓋掉。

IPSI Lab, Yuntech-CSIE

多層次副程式的執行流程

1. **第一層副程式呼叫**：CALL 指令呼叫副程式 `f1`，將返回位址保存到 LR 並跳轉到 `f1`。
2. **第二層副程式呼叫**：在 `f1` 中再次呼叫 `f2`，在呼叫 `f2` 之前，將 LR 保存到堆疊，以防止返回位址被覆蓋。
3. **返回流程**：
    i. 當 `f2` 完成執行後，使用 POP 指令恢復 `f1` 中的返回位址，然後使用 RET 指令返回 `f1`。
    ii. 當 `f1` 完成後，`f1` 從堆疊取回原 LR，再次通過 RET 指令返回主程式。
    iii. 這樣可確保呼叫鏈中的每一層都能正確返回。

多層次副程式呼叫的執行流程圖



IPSI Lab, Yuntech-CSIE

階段	動作	堆疊內容	備註
1	主程式呼叫 `f1`	（空）	LR = 回主程式位置
2	`f1` 呼叫前 PUSH LR	LR(f1)	保存返回主程式位址
3	`f1` 呼叫 `f2`	LR(f1)	CALL `f2` 將新 LR 覆蓋暫存器
4	`f2` 執行完畢後 RET	LR(f1)	返回 `f1`
5	`f1` POP LR 並 RET	（空）	恢復返回主程式位置並回主程式

- 每一層副程式都必須保存自己的 **返回位址（LR）**。
- 透過 **堆疊（Stack）** 可確保多層呼叫時不會覆蓋彼此的返回資訊。
- 返回過程會以 **後進先出（LIFO）** 的方式依序回到上一層。

IPSI Lab, Yuntech-CSIE

組合語言

```
    LD R2, x        ; 將變數 x 載入暫存器 R2
    PUSH R2         ; 將 R2 (x 的值) 推入堆疊
    CALL f1         ; 呼叫副程式 f1
    ST R1, y        ; 將結果 R1 儲存到 y
    RET             ; 返回主程式

x: WORD 1           ; 定義變數 x，初始值為 1
y: RESW 1           ; 定義變數 y，保留一個字的空間

f1:                 ; 定義副程式 f1
    POP R2          ; 從堆疊彈出 R2 (取得之前推入的變數 x 的值)
    PUSH LR         ; 保存 LR 到堆疊，用於返回時恢復呼叫位置
    ST R2, t        ; 將 R2 的值儲存到變數 t
    LD R3, pt       ; 將變數 pt 的位址載入 R3 (R3 現在指向變數 t)
    PUSH R3         ; 將 R3 (指向 t 的位址) 推入堆疊
    CALL f2         ; 呼叫副程式 f2
    ST R1, b        ; 將 f2 返回的結果 R1 儲存到變數 b
    ADD R1, R1, R1 ; 將 R1 自加，這相當於 R1 乘 2
    POP LR          ; 從堆疊彈出 LR，恢復返回位址
    RET             ; 返回上一層

t: RESW 1           ; 定義變數 t，保留一個字的空間
b: RESW 1           ; 定義變數 b，保留一個字的空間
pt: WORD t          ; 定義指向變數 t 的指標

f2:                 ; 定義副程式 f2
    POP R2          ; 從堆疊彈出 R2 (獲取指向變數 t 的位址)
    LD R3, [R2]     ; 將 R2 指向的位址的值 (即變數 t 的值) 載入 R3
    LDI R4, 5       ; 將常數 5 載入 R4
    ADD R1, R3, R4; 將 R3 (t 的值) 與 R4 (5) 相加，結果存入 R1
    ST R1, r        ; 將計算結果 R1 儲存到變數 r
    RET             ; 返回呼叫此副程式的位置

r: RESW 1           ; 定義變數 r，保留一個字的空間
```

C

```c
int main() {
    int x = 1;
    int y;
    y = f1(x);
    return 1;
}

int f1(int t) {
    int b = f2(&t);
    return b + b;
}

int f2(int *p) {
    int r = *p + 5;
    return r;
}
```

Ipsi Lab, Yuntech-CSIE

▶範例 3.12 多層副程式的呼叫的組合語言程式

(a) 組合語言

```
   LD R2, x
   PUSH R2
   CALL f1
   ST R1, y
   RET
x: WORD 1
y: RESW 1

f1:
   POP R2
   PUSH LR
   ST R2, t
   LD R3, pt
   PUSH R3
   CALL f2
   ST R1, b
   ADD R1, R1, R1
   POP LR
   RET
t:RESW 1
b:RESW 1
pt:WORD t

f2:
   POP R2
   LD R3, [R2]
   LDI R4, 5
   ADD R1, R3, R4
   ST R1, r
   RET
r: RESW 1
```

(b) C 語言 (對照版)

```
R2 = x;
push R2; //push parameter x
f1();
y = R1; // y = f1(x)
return;
int x = 1;
int y;

f1() {
  POP R2; // R2=x;
  PUSH LR //保留連結暫存器
  t = R2;
  R3 = pt;
  PUSH R3; // push &t
  f2();
  b = R1; // b = f2(&t)
  R1 = R1 + R1; // = b + b;
  POP LR // 回復連結暫存器
  return; // return b+b;
  int t;
  int b;
  int *pt = &t;
}
f2() {
  POP R2; // R2=p
  R3=[R2]; // R3 = *p
  R4 = 5;
  R1 = R3 + R4; // R1=*p+5
  r = R1; // r = *p+5
  return;
  int r;}
```

(c) C 語言 (真實版)

```
int main() {
  int x = 1;
  int y;
  y = f1(x);
  return 1;
}
int f1(int t) {
  int b = f2(&t);
  return b+b;
}
int f2(int *p) {
  int r= *p+5;
  return r;
}
```